

EDA, Pandas, Visualization

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 1, Day 03, Lecture

21 September 2026

1. Big Picture and Motivation
2. Learning Outcomes
3. Pandas
4. Data Cleaning
5. Visualization and the EDA Workflow
6. Olympiad Practice
7. Limitations and What Comes Next
8. Summary
9. References

- ▶ **Saturday:** Python: lists, loops, dictionaries, functions, objects.
- ▶ **Sunday:** vectors and matrices; NumPy arrays hold the numbers of a dataset.
- ▶ **Today: pandas** holds the whole table (numbers, names, labels, together, with column names), we **clean** it, and we **look** at it with plots. This is the first hour of every Olympiad task.
- ▶ **Rest of the week:** self-study on objects (Tue), NumPy (Wed), cleaning practice (Thu); exam and lab exercise (Fri).

Yesterday's Iris load: numbers in one array, species in another, columns known only by position.

```
X = np.genfromtxt(file, delimiter=",", skip_header=1,
                  usecols=(1, 2, 3, 4))          # numbers only
y = np.genfromtxt(file, delimiter=",", skip_header=1,
                  usecols=5, dtype=str)         # labels apart
X[:, 2].mean()                                # which column was 2 again?
```

On a real table this breaks

The Titanic table has 891 rows and 12 columns: names, sexes, ages, fares, ticket codes, mixed together. **177 ages are missing** and 687 cabin entries are empty. `genfromtxt` gives `nan` for every text cell, the mean of a column with a missing value is `nan`, and “column 5” tells you nothing.

In one sentence

A **DataFrame** is a table with **named columns** of any type; pandas gives one call for each question (missing? duplicates? counts? means per group?) and one call for each plot.

Where you will meet it

Every Friday notebook starts the same way: `read_csv`, `head`, `info`, `isna().sum()`, `value_counts`, a histogram. Thirty minutes of looking saves hours of wrong modelling. Thursday's deck is all about this.

By the end of this session, you will be able to:

- ▶ **Load** a CSV into a DataFrame and describe it: shape, columns, types, first rows.
- ▶ **Select** columns and filter rows, and compute counts and means per group.
- ▶ **Find and fix** missing values, duplicated rows and numbers stored as text.
- ▶ **Choose** the right plot for a question and draw it with matplotlib.
- ▶ **Run** the EDA checklist on a new dataset before touching any model.

EDA: Pandas

Definition

A **DataFrame** is a table: rows are examples, columns have **names** and each column has one type. One column on its own is a **Series**. The row labels on the left are the **index**.

	name	round1	round2	passed
0	Ali	78	82	True
1	Sara	92	NaN	True
2	Omar	85	79	True
3	Lina	64	70	False

- ▶ Yesterday's matrix, plus names for the columns and room for text and booleans.
- ▶ `df["round1"]` is a Series: a NumPy array with a name and an index.
- ▶ NaN (“not a number”) marks a **missing** value. Sara skipped round 2.
- ▶ Under the hood, every numeric column is a NumPy array: `.mean()`, `.std()` work as yesterday.

```
import pandas as pd
df = pd.read_csv("Titanic-Dataset.csv")    # one call, any CSV
df.shape      # (891, 12)  rows, columns
df.head(3)    # the first three rows, as a table
df.columns    # ['PassengerId', 'Survived', 'Pclass', 'Name', ...]
df.dtypes     # int64 / float64 / object (object = text)
df.info()     # columns, non-missing counts, types
```

- ▶ These five calls are the first cell of every task. `info()` already shows the missing values: Age 714 non-null of 891.
- ▶ `df` is an object (Saturday): the dot gives you its attributes (`shape`) and methods (`head()`).

```
df["Age"] # one column (a Series)
df[["Name", "Age"]] # two columns: double brackets
df["Age"].mean() # 29.7 NaN is skipped
df[df["Age"] < 18] # rows where True: 113 children
df[df["Sex"] == "female"].shape # (314, 12)
df.loc[0, "Name"] # one cell: row 0, column "Name"
```

- ▶ `df["Age"] < 18` is a Series of True/False, one per row; putting it inside `df[...]` keeps the True rows. Same idea as Saturday's if inside a loop, in one line for all rows.
- ▶ Combine conditions with `&` and `|`, each in brackets: `df[(df["Sex"] == "female") & (df["Age"] < 18)]`.

```
df["Survived"].value_counts()          # 0: 549, 1: 342
df["Survived"].mean()                  # 0.384   share of ones
df.groupby("Sex")["Survived"].mean()    # female 0.742, male 0.189
df.groupby("Pclass")["Age"].mean()      # 38.2, 29.9, 25.1
df.describe()                          # count, mean, std, min, max
```

groupby = split, apply, combine

Split the rows by the values of one column (three classes), apply a function to each group (mean of Age), combine the answers into a table. Yesterday's centroids were `iris.groupby("Species").mean()`: one line instead of two loops.

EDA: Data Cleaning

	name	round1	round2
0	Ali	78	82
1	Sara	92	NaN
2	Omar	85	79
3	Omar	85	79
4	Lina	64	n/a
5	Yara	91	95

- ▶ **Missing value:** row 1, NaN. A mean over the column ignores it; a model cannot.
- ▶ **Duplicate row:** rows 2 and 3 are the same student twice. Every count is off by one.
- ▶ **Number stored as text:** "n/a" in row 4 turns the whole column into object; `.mean()` fails.

The rule

Look first (`info`, `isna().sum()`, `duplicated().sum()`, `dtypes`), then fix, then look again.

```
df.isna().sum()           # Age 177, Cabin 687, Embarked 2
df = df.drop(columns=["Cabin"]) # 77% missing: drop the column
median = df["Age"].median()  # 28.0
df["Age"] = df["Age"].fillna(median) # fill 177 ages with 28
df = df.dropna(subset=["Embarked"]) # 2 rows: just drop them
df.isna().sum().sum()      # 0
```

- ▶ **Worked example:** round 2 scores 82, 79, 70, 95 with one missing. Mean = $326/4 = 81.5$; fill Sara's NaN with 81.5.
- ▶ **Drop** when few rows (or most of a column) are missing. **Fill** with the median (robust to outliers) or the mean when many rows would be lost.
- ▶ Never fill before looking: 687 of 891 cabins are missing, so filling would invent 77% of the column.

```
df.duplicated().sum()          # rows that repeat an earlier row
df = df.drop_duplicates()      # keep the first copy of each

df["round2"].dtype             # object: text got in
df["round2"] = pd.to_numeric(df["round2"], errors="coerce")
df["round2"].dtype             # float64: "n/a" became NaN
df["Pclass"] = df["Pclass"].astype(str)  # a category, not a size
```

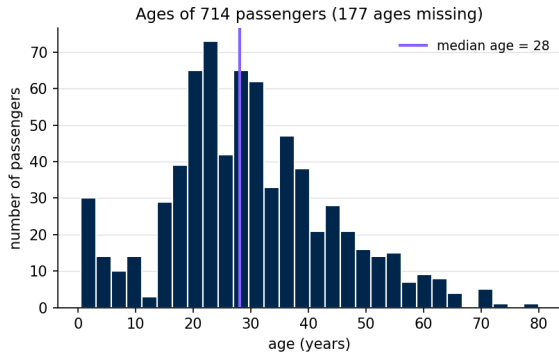
- ▶ `errors="coerce"` turns every unreadable value into NaN, which you then treat as missing (previous slide).
- ▶ **Skipped on purpose:** outliers, inconsistent spellings ("Male" vs "male"), date columns, and encoding text as numbers. Thursday and Week 2.

EDA: **Visualization** and the EDA Workflow

Question	Plot	Call
How is one number spread?	histogram	<code>ax.hist(df["Age"])</code>
How does a value differ between groups?	bar chart	<code>ax.bar(names, values)</code>
Are two numbers related?	scatter plot	<code>ax.scatter(df["x"], df["y"])</code>
How does a value change over time or order?	line plot	<code>ax.plot(x, y)</code>

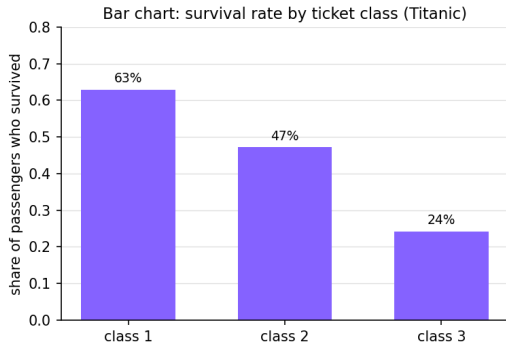
- ▶ A number in a table hides its shape; a plot shows it. The mean age is 29.7, but is it a bump or two bumps?
- ▶ One question per plot. Title, axis labels with units, and a legend when there are groups. Nothing decorative.

Histogram: The Shape of One Column



Source: `scripts/day03_eda_figures.py`, Titanic data set (Kaggle yasserh/titanic-dataset).

- ▶ The axis is cut into **bins**; the bar height is how many rows fall in each bin.
- ▶ Most passengers are 20 to 40; a small bump of babies and children; a long tail to 80.
- ▶ The median (28) sits in the biggest bump: a good value to fill the 177 missing ages with.



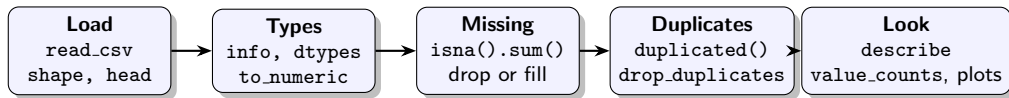
Source: `scripts/day03_eda_figures.py`, Titanic data set.

- ▶ The data behind it is one groupby:
`df.groupby("Pclass")`
`["Survived"].mean()` gives 0.63, 0.47, 0.24.
- ▶ Read it in one glance: first class survived at more than twice the rate of third class.
- ▶ This is the plot you draw for every categorical column against the target on Friday.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(6, 4))      # one figure, one axes
ax.hist(df["Age"].dropna(), bins=30)        # the plot
ax.set_title("Passenger ages")             # always: title, labels
ax.set_xlabel("age (years)")
ax.set_ylabel("number of passengers")
plt.show()

rate = df.groupby("Pclass")["Survived"].mean() # data first
fig, ax = plt.subplots()
ax.bar(rate.index.astype(str), rate.values)    # then plot
```

Same three lines every time: make axes, draw, label. Scatter plots (`ax.scatter(x, y)`) were yesterday's flower picture.



- ▶ **EDA** (exploratory data analysis) is this checklist, in this order, before any model. Write the answers down as you go.
- ▶ The last box asks the real questions: is the target balanced (value_counts)? which columns move with it (groupby, bar chart)? what is the range of each number (describe, histogram)?
- ▶ Lab 2 runs the whole checklist on the Titanic table. Thursday's deck is the tricks and mistakes of each box.

EDA: Olympiad Practice

Task

You receive `train.csv` with a target column `label` and several feature columns. Before modelling, report: (1) the number of rows after removing duplicated rows, (2) the share of each class in `label`, (3) the mean of the column `income` per class, where `income` contains a few entries written as "unknown".

- ▶ Scoring: the three answers are checked exactly. A wrong row count makes every later number wrong.
- ▶ Typical trap: `income` loads as text because of "unknown"; `df["income"].mean()` then raises an error or, worse, is skipped.

Adapted from: NEOAI 2025, "Tricy Table" (Kaggle), whose first hurdle is understanding a table with traps in it.

Recognise: the EDA checklist, boxes 2 to 5. **Check:** row counts before and after; class shares add to 1; the means are plausible numbers, not NaN.

```
df = pd.read_csv("train.csv")
df = df.drop_duplicates() # (1)
n_rows = len(df)
shares = df["label"].value_counts(normalize=True) # (2) shares
df["income"] = pd.to_numeric(df["income"], errors="coerce")
means = df.groupby("label")["income"].mean() # (3) NaN
        skipped
print(n_rows, shares.round(3).to_dict(), means.round(1).to_dict())
```

Every line is one box of the workflow. Six lines; the loops of Saturday would need forty.

EDA: Limitations & What Comes Next

Where it breaks and the mistakes to watch for

- ▶ **Filling hides the problem.** A filled age is a guess; 177 guesses change every mean. Note what you filled and why.
- ▶ **Forgetting to assign:** `df.drop_duplicates()` returns a new table; without `df = ...` nothing changes.
- ▶ **Single versus double brackets:** `df["Age"]` is a Series, `df[["Age"]]` a DataFrame; methods differ.
- ▶ **Cleaning the test set differently from the training set.** Same steps, same fill values, both tables.
- ▶ **Tuesday, Day 04 (self-study):** objects in depth, and DataFrame as an example of a class.
- ▶ **Thursday, Day 06 (Olympiad practice):** the tricks and mistakes of every box of the EDA workflow, tied to Friday's exercise.
- ▶ **Week 2:** the cleaned table goes into a model; `groupby` means become the baseline to beat.

EDA: Summary

- ▶ A **DataFrame** is a table with named, typed columns; `read_csv`, `shape`, `head`, `info` describe it in four calls.
- ▶ **Select** with `df["col"]`, **filter** with a condition inside `df[...]`, **summarise** with `value_counts`, `describe` and `groupby`.
- ▶ **Clean** in a fixed order: types (`to_numeric`), missing values (`isna`, `drop` or `fill`), duplicates (`drop_duplicates`), then look again.
- ▶ **Plot** one question at a time: histogram for spread, bar chart for groups, scatter for two numbers; always title and labels.
- ▶ The **EDA workflow** (load, types, missing, duplicates, look) is the first cell block of every Olympiad notebook.

EDA: References & Resources

- [1] The pandas development team (2026). *10 minutes to pandas*.
https://pandas.pydata.org/docs/user_guide/10min.html

- [2] Hunter, J. D. (2007). Matplotlib: a 2D graphics environment. *Computing in Science & Engineering* 9(3). Tutorial:
<https://matplotlib.org/stable/tutorials/pyplot.html>

- [3] Kaggle (2021). *Titanic Dataset* (yasserh/titanic-dataset), 891 passengers; used for the figures and Lab 2.
<https://www.kaggle.com/datasets/yasserh/titanic-dataset>

- [4] Fisher, R. A. (1936). *Iris* data set, Kaggle copy uciml/iris.
<https://www.kaggle.com/datasets/uciml/iris>

- [5] NEOAI 2025. *Tricy Table* competition on Kaggle; the practice task is adapted from its opening data-reading step.
<https://www.kaggle.com/competitions/neoai-2025-tricy-table-data>
- [6] Altwaim, S. (2026). *Day 2: Data Handling & Visualization*, KAUST Academy & Mawhiba IOAI 2026 training. Missing-value and plot-choice examples adapted from it.